DWM Practical Exam

Simple Python Code Handbook

11 matched experiments | Large 15 pt code

Ctrl+F: search a topic or Experiment 8, for example

01 ETL and Data Warehouse

02 OLAP Operations

03 Data Preprocessing and EDA

04 Customer Discretization

05 Disease Prediction using KNN

06 E-commerce Decision Tree

07 Classification, Clustering and Association

08 Telecom K-Means Clustering

09 Hierarchical Clustering

10 Apriori Association Rules

11 PageRank

Use the filenames and column names required by your actual dataset.

Teams 1-2 require Excel; Teams 7 specifies WEKA/R. Their Python equivalents are included for practice.

Teams 12 is Power BI. No corresponding Python code was present in your supplied PDF.

All sections ran successfully on example datasets. Your exam data must meet the requirements on each page.

Find an experiment

Press Ctrl+F and enter a topic or experiment number.

Examples: Experiment 8, Exp8, KMeans, K-Means, KNN.

Click any entry below to jump directly to its code.

Experiment 1 - ETL and Data Warehouse
ETL | Data Warehouse | Star Schema | Snowflake

Experiment 2 - OLAP Operations
OLAP | Power Pivot | Roll-up | Drill-down | Slice | Dice

Experiment 3 - Data Preprocessing and EDA
Preprocessing | Pre-processing | EDA | Diabetes

Experiment 4 - Customer Discretization
Discretization | Binning | Customer Segmentation

Experiment 5 - Disease Prediction using KNN
KNN | K-Nearest Neighbors | Disease Prediction

Experiment 6 - E-commerce Decision Tree
Decision Tree | DecisionTree | E-commerce | Purchase

Experiment 7 - Classification and Clustering (Part A)
WEKA | R | Classification | Clustering | Association

Experiment 7 - Association Rules (Part B)
WEKA | R | Classification | Clustering | Association

Experiment 8 - Telecom K-Means Clustering
KMeans | K-Means | K Means | Telecom | Clustering

Experiment 9 - Hierarchical Clustering
Hierarchical | Agglomerative | Dendrogram | Single Linkage

Experiment 10 - Apriori Association Rules
Apriori | Association Rules | Market Basket

Experiment 11 - PageRank
PageRank | Page Rank | Web Page Ranking

Bookmarks in the PDF sidebar provide the same shortcuts.

Before you run the code

Keep the CSV and Python file in the same folder. Match column names exactly, including capitals and spaces.

Remove customer, sample, or other ID columns from model features. IDs identify records; they do not measure behaviour.

For K-means here, use at least 2 numeric features for the scatter plot and at least 3 distinct feature rows for 3 clusters.

Example: 2 numeric features, 3 distinct rows

ID	MonthlyBill	DataUsage
C1	300	10
C2	500	25
C3	800	50
C4	300	10

C1 and C4 have the same feature values, so they count as one distinct feature row. CustomerID does not count as a feature.

Copying the code

Select only the code block. Indentation uses four ordinary spaces; there are no line numbers or wrapped code lines.

PDF viewers handle copying differently. The companion ZIP contains identical .py files and preserves exact indentation.

Experiment 7 has two standalone parts. Run Part A for classification and clustering, and Part B for association rules.

```
Packages: pandas matplotlib scikit-learn scipy
          mlxtend networkx
```

Experiment 1 | Exp 1 | Exp1 | Experiment 01

ETL and Data Warehouse

File: sales.csv. Columns: OrderID, Product, Category,
Customer, Region, Sales. Sales must be numeric.
Python practice equivalent; Teams requires Excel ETL.

```
import pandas as pd

df = pd.read_csv("sales.csv")
df = df.drop_duplicates().dropna()

products = df[["Product", "Category"]]
products = products.drop_duplicates()
customers = df[["Customer", "Region"]]
customers = customers.drop_duplicates()
facts = df[["OrderID", "Product", "Customer", "Sales"]]

print("Products:", products, sep="\n")
print("Customers:", customers, sep="\n")
print("Facts:", facts.head(), sep="\n")
print(df.groupby("Region")["Sales"].sum())
```

Search: ETL | Data Warehouse | Star Schema | Snowflake

Experiment 2 | Exp 2 | Exp2 | Experiment 02

OLAP Operations

File: sales.csv. Columns: Region, Category, Product, Sales.
Replace East and Electronics with values in your CSV.
Python practice equivalent; Teams requires Excel Power Pivot.

```
import pandas as pd

df = pd.read_csv("sales.csv")

# Roll-up
print(df.groupby("Region")["Sales"].sum())

# Slice
print(df[df["Region"] == "East"])

# Dice
print(df[(df["Region"] == "East") &
         (df["Category"] == "Electronics")])

# Drill-down
print(df.groupby(["Region", "Category", "Product"])
      ["Sales"].sum())

# Pivot
print(df.pivot_table(
    values="Sales", index="Region",
    columns="Category", aggfunc="sum", fill_value=0
))
```

Search: OLAP | Power Pivot | Roll-up | Drill-down | Slice | Dice

Experiment 3 | Exp 3 | Exp3 | Experiment 03

Data Preprocessing and EDA

File: diabetes.csv. Outcome is the target column.
Other numeric columns are features; exclude numeric IDs.
Each feature needs at least one non-missing value.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("diabetes.csv").drop_duplicates()
X = df.drop(columns="Outcome").select_dtypes("number")

print("Missing values:", X.isna().sum(), sep="\n")
X = X.fillna(X.median()).astype(float)

for col in X.columns:
    q1, q3 = X[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    X[col] = X[col].clip(
        q1 - 1.5 * iqr, q3 + 1.5 * iqr
    )

print(X.describe())
X.hist(figsize=(10, 7))
plt.tight_layout()
plt.show()

X = pd.DataFrame(
    StandardScaler().fit_transform(X),
    columns=X.columns, index=X.index
)
print("Scaled data:", X.head(), sep="\n")
```

Search: Preprocessing | Pre-processing | EDA | Diabetes

Experiment 4 | Exp 4 | Exp4 | Experiment 04

Customer Discretization

File: customers.csv. Numeric columns: Age and
Annual Income (k$). Ages must be positive.
Age groups and income bins are examples; adjust if needed.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("customers.csv").dropna()

df["Age_Group"] = pd.cut(
    df["Age"], bins=[0, 25, 40, 60, float("inf")],
    labels=["Young", "Adult", "Middle", "Senior"]
)
df["Income_Group"] = pd.cut(
    df["Annual Income (k$)"],
    bins=[-float("inf"), 40, 80, float("inf")],
    labels=["Low", "Medium", "High"]
)

print(df[["Age", "Age_Group", "Income_Group"]].head())
table = pd.crosstab(df["Age_Group"], df["Income_Group"])
print(table)
table.plot(kind="bar")
plt.tight_layout()
plt.show()
```

Search: Discretization | Binning | Customer Segmentation

Experiment 5 | Exp 5 | Exp5 | Experiment 05

Disease Prediction using KNN

File: diabetes.csv. Outcome is a categorical target.
All remaining columns must be numeric features; remove IDs.
Need 5+ training rows; each class needs enough rows to split.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

df = pd.read_csv("diabetes.csv").dropna()
X = df.drop(columns="Outcome")
y = df["Outcome"]

a, b, c, d = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
a = scaler.fit_transform(a)
b = scaler.transform(b)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(a, c)
pred = model.predict(b)

print("Accuracy:", accuracy_score(d, pred))
print("Confusion matrix:", confusion_matrix(d, pred))
```

Search: KNN | K-Nearest Neighbors | Disease Prediction

Experiment 6 | Exp 6 | Exp6 | Experiment 06

E-commerce Decision Tree

File: ecommerce.csv. Columns: UserID, Purchase, and features.
Purchase is the target; text features are one-hot encoded.
Each class needs enough rows for the stratified split.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score

df = pd.read_csv("ecommerce.csv").dropna()
X = pd.get_dummies(df.drop(columns=["UserID", "Purchase"]))
y = df["Purchase"]

a, b, c, d = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(a, c)
print("Accuracy:", accuracy_score(d, model.predict(b)))

plt.figure(figsize=(14, 8))
plot_tree(model, feature_names=X.columns, filled=True)
plt.show()
```

Search: Decision Tree | DecisionTree | E-commerce | Purchase

Experiment 7 | Exp 7 | Exp7 | Experiment 07 | Part A

Classification and Clustering

File: data.csv. Class is the target; other columns numeric.
Remove IDs. For K-means, need 3+ distinct feature rows.
Python practice equivalent; Teams specifies WEKA/R.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

df = pd.read_csv("data.csv").dropna()
X = df.drop(columns="Class")
y = df["Class"]

# Classification
a, b, c, d = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = DecisionTreeClassifier(random_state=42)
model.fit(a, c)
print("Accuracy:", accuracy_score(d, model.predict(b)))

# Clustering
Z = StandardScaler().fit_transform(X)
labels = KMeans(
    n_clusters=3, n_init=10, random_state=42
).fit_predict(Z)
print("Cluster sizes:", pd.Series(labels).value_counts())
```

Search: WEKA | R | Classification | Clustering | Association

Association Rules

File: transactions.csv. Columns: TransactionID and Item.
One purchased item per row; IDs repeat within each basket.
Example: T1,Milk and T1,Bread belong to the same basket.

```
import pandas as pd
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules

df = pd.read_csv("transactions.csv").dropna()
basket = pd.crosstab(df["TransactionID"], df["Item"])
basket = basket.astype(bool)

items = apriori(
    basket, min_support=0.05, use_colnames=True
)
rules = association_rules(
    items, metric="confidence", min_threshold=0.3,
    num_itemsets=len(basket)
)
print(items)
print(rules[["antecedents", "consequents", "lift"]])
```

Experiment 8 | Exp 8 | Exp8 | Experiment 08

Telecom K-Means Clustering

File: telecom.csv. Exclude CustomerID from numeric features.
Need 2+ numeric features for this plot and 3+ distinct rows.
Distinct rows ignore IDs: (300,10), (500,25), (800,50).

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

df = pd.read_csv("telecom.csv").dropna()
X = df.drop(columns="CustomerID").select_dtypes("number")
Z = StandardScaler().fit_transform(X)

df["Cluster"] = KMeans(
    n_clusters=3, n_init=10, random_state=42
).fit_predict(Z)

print(df.groupby("Cluster")[X.columns].mean())

plt.scatter(Z[:, 0], Z[:, 1], c=df["Cluster"])
plt.xlabel(X.columns[0])
plt.ylabel(X.columns[1])
plt.title("K-Means Clustering")
plt.show()
```

Search: KMeans | K-Means | K Means | Telecom | Clustering

Experiment 9 | Exp 9 | Exp9 | Experiment 09

Hierarchical Clustering

File: bio.csv. SampleID is an ID; other features numeric.
Need 3+ samples. Uses single-linkage agglomeration.
maxclust requests at most 3 clusters; ties may yield fewer.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import linkage
from scipy.cluster.hierarchy import dendrogram, fcluster

df = pd.read_csv("bio.csv").dropna()
X = df.drop(columns="SampleID").select_dtypes("number")
Z = StandardScaler().fit_transform(X)

links = linkage(Z, method="single")
dendrogram(links)
plt.title("Hierarchical Clustering")
plt.xlabel("Sample")
plt.ylabel("Distance")
plt.show()

df["Cluster"] = fcluster(links, t=3, criterion="maxclust")
print(df["Cluster"].value_counts())
print(df.groupby("Cluster")[X.columns].mean())
```

Search: Hierarchical | Agglomerative | Dendrogram | Single Linkage

Apriori Association Rules

File: market.csv. Columns: TransactionID and Item.
One item per row: T1,Milk; T1,Bread; T2,Milk; T2,Eggs.
No rules? Lower support/confidence or check the baskets.

```
import pandas as pd
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules

df = pd.read_csv("market.csv").dropna()
basket = pd.crosstab(df["TransactionID"], df["Item"])
basket = basket.astype(bool)

items = apriori(
    basket, min_support=0.05, use_colnames=True
)
rules = association_rules(
    items, metric="confidence", min_threshold=0.3,
    num_itemsets=len(basket)
)

print("Frequent itemsets:", items, sep="\n")
print(rules[["antecedents", "consequents",
             "support", "confidence", "lift"]])
```

Search: Apriori | Association Rules | Market Basket

PageRank

File: links.csv. First column = source; second = target.
Example: A,B means page A links to page B.
Needs nonempty link data; higher scores mean higher rank.

```
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt

df = pd.read_csv("links.csv").dropna()
G = nx.from_pandas_edgelist(
    df, source=df.columns[0], target=df.columns[1],
    create_using=nx.DiGraph()
)

ranks = nx.pagerank(G, alpha=0.85)
for page, score in sorted(
    ranks.items(), key=lambda x: x[1], reverse=True
):
    print(page, round(score, 4))

nx.draw(G, with_labels=True, node_color="lightblue",
        node_size=1500, arrows=True)
plt.title("PageRank")
plt.show()
```

Search: PageRank | Page Rank | Web Page Ranking